

JAKS Inventory ERP

Parallel Coder Briefing & Status Report

Three independent work lanes, runnable as parallel Claude Code sessions, with strict file ownership to guarantee zero merge collisions.

Prepared	2026-05-29
Repo	C:\Users\keith\JAKSInventory\rebuild
Branch baseline	backend/workflow-series-3 (4 commits ahead of main)
Model config	3 lanes · fix blockers first · own-branch-per-lane

1 Where the Project Is

The rebuild runs as three coordinated lanes. The active branch `backend/workflow-series-3` is a clean superset of `main` (4 commits ahead, 0 behind) and mixes backend Workflow Series 3–4 with UI-lane commits. The `feat/document-engine-series` branch is merged to `main` via PR #4.

Decisions locked — 2026-05-29 (Keith)

1. `data/jaks.db` is disposable dev data — **drop & reseed** for Blocker 1; do not preserve.
2. `purchase_orders/detail.html` is **confirmed unused** by `routes/templates/tests` — Backend deletes it.
3. **QBO (Phase M) stays gated** — not started until the UI rollout and core operational flows (PO, invoice, receiving, match, lock, core) are stable.
4. The 13 backend Open Assumptions are **confirmed as written**; do not pause to interview unless one blocks implementation or risks accounting/legal correctness.
5. Macro deviations are the **Architect's call** — default: no new inline-script approvals unless explicitly accepted; restore `preserve_q` if it affects filter behavior; `empty_state` adoption may wait.
6. Workspace drawers → slide-overs is **deferred to the L3 governance pass**; no midstream redesign unless drawers break usability.
7. **Run concurrently, Backend leads slightly** — Backend lands blockers + route contracts first; UI must never build against guessed endpoints.

Lane 1 — Backend (Phases A–O)

Phase	Area	Status
A	Schema foundation — inline migrations, all new models	Complete
B	Inventory + moving-average cost (<code>inventory_service</code>)	Complete
C	Sales-Order fulfillment sources	Partial
D	Tax + locked-invoice enforcement (Series 1)	Complete
E	Payment rewrite (Series 2)	Complete
F	Credit memos (<code>credit_memo_service</code>)	Complete
G	Vendor credits + returns (Series 3)	Complete
H	Cores + locations (Series 2)	Complete
I	Statements + reports (8 report templates live)	Complete
J	Search + optimistic locking	Partial
K	Notifications + audit (<code>notification_service</code> + router)	Complete
L	Communication foundation (<code>messaging_service</code> + 10 templates)	Complete
M	QuickBooks Online push	Deferred — fields exist, no push
N / O	Real messaging providers / serial + kits + merge	Not started

Lane 2 — UI (17-screen rollout + governance)

- **L2 reference lists complete:** Products, Purchase Orders, Invoices.
- **Primitives extracted** — 6 Jinja2 macros in `app/templates/macros/`; 3 lists ported. **Macro library still pending an Architect governance pass.**

- **Queue Boards complete (QB2):** PO Receiving Queue + 3-Way Match Queue.
- **Workspaces:** PO + Invoice are L3-candidates; need a formal L3 governance pass.
- **Not started:** Customer List (L1.5→L2), Quotes alignment, Sales Orders, Vendors, Returns, Payments, Product Detail; Warranty / Cores / Returns queues.

Lane 3 — Document Engine (merged to main)

PO / SO / Core Slip / VCR / Warranty / RA print + PDF generation (`document_render.py`). Stable; not an active lane unless new document types are requested.

2 Pre-Flight: Fix These Blockers First

Per your direction, all three lanes start from a green baseline. These are owned by **Lane B (Backend)** and should land before parallel feature work begins — they take well under a session combined.

BLOCKER 1 — Sales-Order list page returns HTTP 500

GET /sales-orders/ throws `OperationalError: no such column: sales_orders.qbo_so_id`.

Cause: the `SalesOrder` model (`app/models/quote.py`) declares `qbo_so_id`, the `QBOSyncMixin` columns, `payment_mode`, `deposit_amount`, `customer_job_number`, `engine` fields, `ship_to_address_id` and `updated_at` — but `_PENDING_COLUMN_ADDITIONS` in `app/database.py` has **no sales_orders entries**, so existing DBs never get them.

Fix (decided): `data/jaks.db` is disposable — **drop & recreate + reseed per Phase A** (fastest; do not preserve the file). Optionally also add the missing `sales_orders` columns to `_PENDING_COLUMN_ADDITIONS` so any other stale DB self-heals.

Done when: the SO-list smoke test passes — `pytest "tests/test_smoke.py::test_page_200[SO list]"`.

BLOCKER 2 — No test isolation (misleading red suite)

There is no `tests/conftest.py`; tests share one `data/jaks.db`. Files pass individually (series-3 is 20/20) but the full suite cascades to **38 failed / 24 errored** from shared state.

Fix: add a `conftest.py` with a function/module-scoped fixture that builds a fresh in-memory (or temp-file) SQLite DB per test and overrides the FastAPI DB dependency.

Done when: `pytest -q` is green end-to-end with no ordering dependence.

HOUSEKEEPING (Backend lane / you)

Delete `app/templates/purchase_orders/detail.html` — verified unused (the `/ {po_id}` route renders `workspace.html`; no route, include, or test references it).

Prune 3 stale locked worktrees that point at deleted dirs: `git worktree prune` (then `git worktree list` to confirm).

Commit or stash the 2 uncommitted edits (`JAKS_UI_Change_Plan.md`, `app/templates/invoices/list.html`) before branching so lanes fork from a clean tree.

3 Branching & Coordination

Setup (do once, before launching the lanes):

- Commit current work on `backend/workflow-series-3`, then merge it into `main` so every lane forks from one clean baseline.
- Land the two blockers (Section 2) on `main`.
- Create one branch per lane off `main`: `lane/ui-builder`, `lane/backend`, `lane/ui-architect`.

The golden rule of parallel work:

No two lanes may edit the same file in the same session.

Collisions are prevented by **file ownership**, not by luck. Section 4 assigns every file to exactly one lane. The one structural seam — list **routes** (Backend) vs list **templates** (UI) — is handled the way the Invoice List already was: Backend lands the route (tabs, counts, preview endpoint) first; UI builds the template against it. Each lane commits only inside its ownership set. `Rebase on main daily`; integrate via PR. The Architect lane reviews before UI screens are marked complete.

4 The Three Parallel Lanes

Lane	Role	Branch	Owns
A	UI Builder	<code>lane/ui-builder</code>	Screen templates (lists, queues, workspaces)
B	Backend	<code>lane/backend</code>	Services, models, list routes, DB, tests
C	UI Architect	<code>lane/ui-architect</code>	Macros, design system, governance, plan doc

Lane A — UI Builder

Branch: `lane/ui-builder`

Read first (in order):

- Invoke the **jaks-ui-governance** skill — it is the design system and is mandatory before writing any template code.
- `JAKS_UI_Change_Plan.md` — all sections, especially §2 (Operational List), §2A (Queue Board), the Governance Pass Checklist, and the Rollout Order.
- Reference screens: `app/templates/products/list.html` (L2 ref), `purchase_orders/receiving_queue.html` (QB2 ref).

Tasks, in rollout order:

- **Customer List** L1.5→L2 — build from the new macros from the start (dock, bulk toolbar, tabs, stripe). Depends on Lane B providing the `tab/counts` route + `/customers/preview/{id}`.
- **Warranty Queue, Cores Queue, Returns Queue** — QB2; copy `receiving_queue.html`, adapt `state_meta` + `group-by` + `metrics`. Depends on Lane B queue routes.
- **Quotes List** alignment pass — add border-l-4 stripe, preview dock, pb-52.
- **Sales Orders / Vendors / Payments / Returns lists** — L1→L2 full upgrades (remove `tbl-*` classes).

- **Product Detail** — section-based card layout L1→L2.

Lane A — file ownership (edit ONLY these)

app/templates/{customers,warranty,cores,returns,quotes,sales_orders,vendors,payments,products}/*.html (screen templates + _preview_panel partials)

Do NOT edit router .py, service .py, models, macros/, or JAKS_UI_Change_Plan.md. Need a route shape or a new macro? Request it from Lane B / Lane C.

Lane A — done criteria (per screen)

All 11 §2 elements (or 8 §2A elements for queues) present; no tbl-* classes; renders without error; submitted to Lane C for governance — **not** self-marked complete.

Lane B — Backend

Branch: lane/backend

Read first:

- `BACKEND_IMPLEMENTATION_PLAN.md` — the source of truth; match every task to its priority phase, do not skip ahead.
- `app/database.py` (inline migration mechanism), `app/services/base.py` (permission + optimistic lock helpers).

Tasks:

- **FIRST — both Section 2 blockers** (SO-list columns, confest isolation). Gate the other lanes on these.
- **Finish Phase C** — SO fulfillment-source state machine + linked-PO FIFO allocation on receipt.
- **Finish Phase J** — wire the optimistic-lock version check into all financial save paths; SearchService ranking + phone normalization + combined OEM result.
- **Provide list routes for Lane A screens** — for each new L2 list, add the `tab` param + unfiltered counts + `/[{resource}]/preview/{id}` endpoint (registered *before* `/[{id}]`), mirroring `invoices.py`.
- **Phase M (QBO) — DO NOT START.** Gated until the UI rollout + core operational flows (PO, invoice, receiving, match, lock, core) are stable. Leave the existing `qbo_*` fields dormant.

Lane B — file ownership

`app/services/*.py`, `app/models/*.py`, `app/database.py`, `app/constants.py`, `tests/*`, and the **route logic** (Python view functions) in `app/routers/*.py`.

Seam rule: you may add/modify the route function, but do not restyle its template — that is Lane A. Touch only the `.py` side.

Lane B — done criteria

Each phase passes its plan-defined Gate; new/changed code has a test in `tests/`; full suite green; no model column exists without a matching migration entry.

Lane C — UI Architect (Pattern Owner / Governance)

Branch: lane/ui-architect

Read first:

- The **jaks-ui-governance** skill and the full `JAKS_UI_Change_Plan.md` (you own this document).

Tasks:

- **Clear the pending governance backlog:** (1) macro-library pass — rule on the 3 flagged deviations per owner default: *do not* approve the inline-script P2 pattern unless you explicitly accept it; **restore preserve_q** if it affects expected filter behavior; `empty_state` adoption may wait if not blocking. (2) Invoice Workspace L3 pass; (3) PO Workspace L3 pass — at this pass, decide the drawers→slide-over question (deferred until here; no midstream redesign).
- **Own the macro library** — when Lane A needs a new primitive, you build / approve it in `app/templates/macros/`.

- **Run a governance pass** on every Lane A screen before it is marked complete; punch specific issues, do not redesign.
- **Keep the plan current** — update the maturity table + Rollout Order and As-Built records as screens land.

Lane C — file ownership

app/templates/macros/*.html, JAKS_UI_Change_Plan.md, and base/shared shells
(app/templates/base.html slide-over / modal / Ctrl+K).

Reviews Lane A output but edits screens only to correct an architectural defect (per the governance role).

Lane C — done criteria

Pending governance items closed; macro library approved; every completed-marked screen has a recorded governance pass in the plan.

5 Collision Matrix (Who Owns What)

If a path is not listed for a lane, that lane must not edit it. The only shared seam is router files, split by language: Python view logic (B) vs the template it renders (A).

Path	Owner
app/templates/{screens}/*.html (lists, queues, workspaces)	Lane A
app/templates/macros/*.html	Lane C
app/templates/base.html (shared shells)	Lane C
JAKS_UI_Change_Plan.md	Lane C
app/routers/*.py — Python view functions / route logic	Lane B
app/services/*.py	Lane B
app/models/*.py · app/database.py · app/constants.py	Lane B
tests/*	Lane B
BACKEND_IMPLEMENTATION_PLAN.md	Lane B

Dependency handshakes (sequence these)

New L2 list: Lane B lands route (tab+counts+preview) → Lane A builds template → Lane C governance pass.

New queue board: Lane B lands queue route → Lane A copies receiving_queue.html → Lane C pass.

New macro needed: Lane A requests → Lane C builds/approves in macros/ → Lane A consumes.

6 How to Launch Each Lane

Open three Claude Code sessions in the repo, each checked out to its lane branch. Paste the matching kickoff prompt. Each prompt is self-contained. **Run concurrently, but stagger the start: launch Lane B first** and let it land the two blockers + the route contracts each UI screen needs; then bring up Lanes A and C, which follow the dependency handshakes in §5. UI must never build against guessed endpoints.

Lane A kickoff prompt

You are the UI Builder for JAKS Inventory. First invoke the jaks-ui-governance skill and read JAKS_UI_Change_Plan.md in full. You own ONLY screen templates under app/templates/<screen>/ – never edit routers, services, models, macros, or the plan doc. Build the next Rollout Order screen (start: Customer List L1.5→L2) using the macros in app/templates/macros/. Confirm the list route + preview endpoint exist (ask Lane B if not). Submit each screen for governance; do not self-mark complete. Branch: lane/ui-builder.

Lane B kickoff prompt

You are the Backend coder for JAKS Inventory. Read BACKEND_IMPLEMENTATION_PLAN.md. FIRST fix two blockers: (1) add the missing sales_orders columns (qbo_so_id, QBOSyncMixin fields, payment_mode, deposit_amount, engine/job/ship fields, updated_at) to _PENDING_COLUMN_ADDITIONS in app/database.py so GET /sales-orders/ stops 500ing; (2) add tests/conftest.py giving each test an isolated SQLite DB so the full suite is green. Then continue Phases C and J and provide L2 list routes on request. You own services, models, db, tests, and router view logic only – not templates. Branch: lane/backend.

Lane C kickoff prompt

You are the UI Architect for JAKS Inventory. Invoke the jaks-ui-governance skill; you own JAKS_UI_Change_Plan.md and app/templates/macros/. Clear the pending governance backlog: (1) macro-library pass incl. the 3 flagged deviations; (2) Invoice Workspace L3 pass; (3) PO Workspace L3 pass. Then review each Lane A screen before it is marked complete and keep the plan's maturity table current. Punch specific issues; do not redesign. Branch: lane/ui-architect.

Open questions for Keith are listed in the chat alongside this document. This PDF is generated from JAKS_UI_Change_Plan.md, BACKEND_IMPLEMENTATION_PLAN.md, the git history, and a live test run on 2026-05-29.